UNIVERSIDAD NACIONAL DEL ALTIPLANO

Escuela Profesional de Ingeniería de Sistemas

GUÍA DE PRÁCTICA DE LABORATORIO

SEMANA 2

Abstracción y Encapsulamiento Avanzado — Invariantes, Propiedades y Diseño por Responsabilidades

Semestre 2026-II · Continuación del Proyecto Integrador

Curso:	Programación Orientada a Objetos II — SIS206
Semana:	2 (Encapsulamiento avanzado)
Lenguaje:	Python 3.12+ / C++17
Duración:	2 horas de práctica (laboratorio)
Docente:	Dr. Aldo Hernán Zanabria Gálvez
ORCID / RENACYT:	0000-0003-3314-8768 | P0020608

Puno — Perú — 2026

I. OBJETIVO GENERAL

Diseñar e implementar clases con encapsulamiento estricto mediante invariantes de clase, propiedades con validación (Python) y getters/setters con control de invariantes (C++), aplicando el principio de diseño por responsabilidades para que cada clase tenga una sola razón de cambio — primer paso formal hacia el principio de Responsabilidad Única (SRP) que se estudiará en la Semana 9.

1.1 Objetivos Específicos

- Comprender la diferencia entre encapsulamiento superficial (solo ocultar datos) y encapsulamiento profundo (proteger invariantes de clase).
- Implementar invariantes de clase mediante validaciones en setters y en el constructor.
- Aplicar el decorador @property en Python para crear atributos de solo lectura y de lectura/escritura con validación.
- Implementar getters/setters validados en C++17 con lanzamiento correcto de std::invalid_argument.
- Diseñar clases siguiendo el principio de "una sola responsabilidad" como guía de descomposición.
- Avanzar en el Proyecto Integrador: encapsular correctamente las 3 clases definidas en la Semana 1.

II. MARCO TEÓRICO

2.1 Invariantes de Clase — El Contrato Interno

Un invariante de clase es una condición que debe ser verdadera en todo momento en que el objeto sea visible externamente: antes de cualquier método público y después de él. Los invariantes son la razón fundamental del encapsulamiento: si los atributos son públicos, nada impide que el código externo rompa el invariante (Meyer, 1997):

```
# Sin invariante — el objeto puede quedar en estado inválido class Calificacion:     def __init__(self, valor):         self.valor = valor    # PUBLICO — cualquiera puede escribir -5 o 999  # Con invariante protegido — el objeto SIEMPRE es consistente class Calificacion:     _MIN, _MAX = 0.0, 20.0      def __init__(self, valor: float):         self._valor = None    # inicializar ANTES de llamar al setter         self.valor = valor  # usa el setter con validacion      @property     def valor(self) -> float:         return self._valor      @valor.setter     def valor(self, v: float) -> None:         if not (self._MIN <= v <= self._MAX):             raise ValueError(f"Calificacion {v} fuera de rango [{self._MIN},{self._MAX}]")         self._valor = v      # Invariante: self._valor siempre pertenece a [0.0, 20.0]
```

2.2 Name Mangling en Python — __atributo

Python implementa el ocultamiento mediante convención (_attr = protegido) y mangling (__attr = privado). El doble guion bajo hace que Python renombre el atributo a _NombreClase__attr, dificultando el acceso accidental desde el exterior, aunque no lo impide completamente:

```
class Docente:     def __init__(self, dni: str):         self.__dni = dni    # almacenado como _Docente__dni  d = Docente("12345678") print(d.__dni)          # AttributeError: no existe __dni print(d._Docente__dni)  # "12345678" — accesible, pero no deberías  # Acceso correcto: mediante @property publica # @property # def dni(self):     return self.__dni
```

2.3 Propiedades Calculadas (read-only) en Python

No todo @property necesita un setter. Una propiedad calculada (derivada de otros atributos) debe ser de solo lectura — intentar asignarla directamente debe resultar en AttributeError:

```
class Estudiante:     def __init__(self, nombre: str, apellidos: str):         self._nombre = nombre         self._apellidos = apellidos      @property     def nombre_completo(self) -> str:         """Propiedad CALCULADA — sin setter, de solo lectura."""         return f"{self._apellidos}, {self._nombre}"      @property     def iniciales(self) -> str:         return f"{self._nombre[0]}.{self._apellidos[0]}."  e = Estudiante("Juan", "Mamani Quispe") print(e.nombre_completo)   # "Mamani Quispe, Juan" e.nombre_completo = "otro" # AttributeError — no hay setter
```

2.4 Diseño por Responsabilidades — ¿A quién pertenece este dato?

Una técnica de diseño fundamental es preguntarse: "¿Qué clase tiene la responsabilidad de conocer X o hacer Y?" La respuesta guía la ubicación de atributos y métodos. Una clase mal diseñada tiene atributos que "no le pertenecen" o métodos que trabajan principalmente sobre datos de otra clase (Meyer, 1997):

Pregunta de diseño	Clase responsable
¿Quién conoce el PPA de un estudiante?	Estudiante — porque tiene sus calificaciones.
¿Quién sabe si un libro está disponible?	Libro — porque tiene su propio estado (disponible/prestado).
¿Quién formatea el nombre para imprimir?	No el Docente — una clase Formateador o __str__ separado.

💡 **Conexión con el Proyecto Integrador**
Hoy aplicará encapsulamiento estricto a las 3 clases definidas en la Semana 1. Cada clase debe tener invariantes claros y responsabilidades delimitadas. Este diseño sobrevivirá sin cambios mayores hasta la Semana 7 (modelo UML completo) y la Semana 9 (revisión SOLID).

III. MATERIALES

Herramienta	Uso
Python 3.12+ / Colab	Parte 1: propiedades, invariantes, name mangling
g++ -std=c++17 -Wall	Parte 2: setters validados, const correctness
Draw.io / StarUML	Diagrama de clases actualizado del proyecto (Act. 5)
Git / GitHub	Commit del avance de la Semana 2 al repositorio del proyecto

IV. INSTRUCCIONES PREVIAS

1. Abrir el notebook de Colab de la Semana 1 y agregar una nueva sección "Semana 2".
2. Tener el código de las clases refactorizadas de la Semana 1 disponible como base para esta práctica.
3. Para C++: compilar siempre con g++ -std=c++17 -Wall -Wextra -o prog main.cpp
4. Al terminar, hacer commit de todo el avance al repositorio GitHub del proyecto con el mensaje: "Semana 2: encapsulamiento avanzado e invariantes".

V. DESARROLLO DE ACTIVIDADES

ACTIVIDAD 1 — @property avanzado — invariantes con setters validados (Python)

Python + C++

Implemente la clase Curso del sistema académico con encapsulamiento estricto. El Curso tiene: codigo (string de formato SIS + 3 dígitos), nombre, creditos (1-6), y cupos_maximo (1-50). Todas las validaciones van en los setters correspondientes.

```
# Práctica S2 — Actividad 1: Clase Curso con encapsulamiento estricto import re  class Curso:     _PATRON_CODIGO = re.compile(r"^[A-Z]{2,4}\d{3}$")     _MIN_CREDITOS, _MAX_CREDITOS = 1, 6     _MIN_CUPOS,    _MAX_CUPOS    = 1, 50      def __init__(self, codigo: str, nombre: str,                  creditos: int, cupos_maximo: int):         self.codigo        = codigo         # usa setter         self._nombre       = nombre         self.creditos      = creditos        # usa setter         self.cupos_maximo  = cupos_maximo    # usa setter         self.__inscritos   = 0               # privado — no tiene setter publico      @property     def codigo(self) -> str:         return self.__codigo      @codigo.setter     def codigo(self, valor: str) -> None:         # TODO: validar con _PATRON_CODIGO, lanzar ValueError si no coincide         ...     @property     def creditos(self) -> int:         return self.__creditos     @creditos.setter     def creditos(self, valor: int) -> None:         # TODO: validar rango [1, 6]         ...      @property     def cupos_maximo(self) -> int:         return self.__cupos_maximo      @cupos_maximo.setter     def cupos_maximo(self, valor: int) -> None:         # TODO: validar rango [1, 50]         ...      @property     def inscritos(self) -> int:         """Solo lectura — no tiene setter publico."""         return self.__inscritos      @property     def cupos_disponibles(self) -> int:         """Propiedad CALCULADA — sin setter."""         return self.__cupos_maximo - self.__inscritos      @property     def esta_lleno(self) -> bool:         """Propiedad calculada booleana."""         return self.__inscritos >= self.__cupos_maximo      def inscribir_estudiante(self) -> None:         """Controla el invariante: inscritos <= cupos_maximo."""         if self.esta_lleno:             raise RuntimeError(f"Curso {self.__codigo} sin cupos disponibles")         self.__inscritos += 1      def __str__(self) -> str:         return (f"{self.__codigo} - {self._nombre}"                 f" ({self.__creditos} cr)"                 f" [{self.__inscritos}/{self.__cupos_maximo} inscritos]")  # --- Pruebas ------------------------------------------------------------ c = Curso("SIS206", "POO II", 4, 30) print(c) print(f"Cupos disponibles: {c.cupos_disponibles}")  for _ in range(3):     c.inscribir_estudiante() print(c)  # Invariantes — deben lanzar ValueError: for caso in [("X9", "nombre", 4, 30), ("SIS206", "nombre", 0, 30), ("SIS206", "nombre", 4, 0)]:     try:         Curso(*caso)     except ValueError as ex:         print(f"Error esperado: {ex}")
```

ACTIVIDAD 2 — Propiedades calculadas y encadenamiento (Python)

Python + C++

Implemente la clase Docente que reutiliza Curso mediante composición (una lista de cursos asignados). Las propiedades calculadas deben derivar su valor del estado actual del objeto, no almacenarse como atributos adicionales.

```
# Práctica S2 — Actividad 2: Docente con propiedades calculadas from datetime import
date  class Docente:      _MAX_CURSOS = 3    # invariante: un docente no puede tener > 3
cursos      def __init__(self, dni: str, nombres: str, apellidos: str,
correo: str, fecha_nac: date):          self.__dni       = self._validar_dni(dni)
self.__nombres   = nombres          self.__apellidos = apellidos          self.__correo
= self._validar_correo(correo)          self.__fecha_nac = fecha_nac
self.__cursos: list[Curso] = []      @staticmethod     def _validar_dni(dni: str) ->
str:          # TODO: exactamente 8 digitos          ...      @staticmethod     def
_validar_correo(correo: str) -> str:          # TODO: debe terminar en @unap.edu.pe
...      @property     def nombre_completo(self) -> str:          return
f"{self.__apellidos}, {self.__nombres}"      @property     def edad(self) -> int:
"""Calculada en tiempo real — no se almacena."""          hoy = date.today()
anios = hoy.year - self.__fecha_nac.year          if (hoy.month, hoy.day) <
(self.__fecha_nac.month, self.__fecha_nac.day):              anios -= 1          return
anios      @property     def total_creditos(self) -> int:          """Suma de creditos de
todos los cursos asignados."""          return sum(c.creditos for c in self.__cursos)
@property     def total_inscritos(self) -> int:          """Total de estudiantes
inscritos en todos sus cursos."""          return sum(c.inscritos for c in self.__cursos)
def asignar_curso(self, curso: Curso) -> None:          if len(self.__cursos) >=
self._MAX_CURSOS:              raise RuntimeError(f"{self.nombre_completo} ya tiene
{self._MAX_CURSOS} cursos asignados")          self.__cursos.append(curso)      def
__str__(self) -> str:          return (f"Dr. {self.nombre_completo}"                f" |
Edad: {self.edad}"                 f" | Cursos: {len(self.__cursos)}"                f"
| Total creditos: {self.total_creditos}")  # --- Pruebas ----------------------------
------------------------ from datetime import date doc = Docente("12345678", "Aldo
Hernan", "Zanabria Galvez",              "azanabria@unap.edu.pe", date(1980, 5, 14))
poo2 = Curso("SIS206", "POO II", 4, 30) poo2.inscribir_estudiante()
poo2.inscribir_estudiante() doc.asignar_curso(poo2) print(doc) print(f"Total inscritos
en sus cursos: {doc.total_inscritos}")
```

ACTIVIDAD 3 — Encapsulamiento en C++17 — const correctness y setters validados

Python + C++

Reimplemente la clase Curso en C++17 con: (i) atributos privados con convención de sufijo guion bajo, (ii) setters que lanzan std::invalid_argument, (iii) métodos constantes (const) para todos los getters y métodos de solo lectura.

```
// Practica S2 — Actividad 3: Curso.hpp — encapsulamiento C++17 #pragma once #include
<string> #include <regex> #include <stdexcept> #include <iostream>  class Curso {
std::string codigo_;     std::string nombre_;     int         creditos_;     int
cuposMaximo_;     int         inscritos_ = 0;    // inicializado en declaracion  public:
Curso(std::string codigo, std::string nombre,           int creditos, int cuposMaximo)
: nombre_(std::move(nombre)) {          setCodigo(codigo);          // validar en el
constructor          setCreditos(creditos);          setCuposMaximo(cuposMaximo);     }
// --- Setters validados ----------------------------------------------     void
setCodigo(const std::string& cod) {          // TODO: regex [A-Z]{2,4}[0-9]{3}
codigo_ = cod;     }      void setCreditos(int c) {          if (c < 1 || c > 6)
throw std::invalid_argument("Creditos fuera de rango [1,6]");          creditos_ = c;
}      void setCuposMaximo(int q) {          if (q < 1 || q > 50)              throw
std::invalid_argument("Cupos fuera de rango [1,50]");          cuposMaximo_ = q;     }
// --- Getters (const — no modifican el objeto) ---------------------     const
std::string& getCodigo()  const { return codigo_; }     const std::string& getNombre()
const { return nombre_; }     int  getCreditos()              const { return creditos_;
}     int  getCuposMaximo()           const { return cuposMaximo_; }     int
getInscritos()             const { return inscritos_; }      // --- Propiedades
calculadas (const) ------------------------------     int  cuposDisponibles()
```

```
const { return cuposMaximo_ - inscritos_; }     bool estaLleno()                const {
return inscritos_ >= cuposMaximo_; }      // --- Logica de negocio ----------------------
----------------------     void inscribirEstudiante() {         if (estaLleno())
throw std::runtime_error("Curso sin cupos: " + codigo_);         ++inscritos_;     }
void imprimir() const {         std::cout << codigo_ << " - " << nombre_
<< " (" << creditos_ << " cr)"                  << " [" << inscritos_ << "/" <<
cuposMaximo_ << "]\n";     } };  // main.cpp para prueba // int main() { //     Curso
c("SIS206", "POO II", 4, 30); //     c.inscribirEstudiante(); //     c.imprimir(); //
try { Curso bad("SIS206","bad",0,30); } //     catch(std::invalid_argument& ex){
std::cout << ex.what() << "\n"; } // }
```

ACTIVIDAD 4 — Comparación Python vs C++ — tabla de equivalencias de encapsulamiento

Python + C++

Complete la tabla de equivalencias entre los mecanismos de encapsulamiento de Python y C++17. Luego responda la pregunta de análisis al pie.

Concepto	Python	C++17
Atributo privado	__attr (name mangling)	tipo attr_; acceso: private:
Atributo protegido	_attr (convención)	tipo attr_; acceso: protected:
Getter de solo lectura	@property def attr(self)	const TipoR& getAttr() const
Getter + setter validado	@attr.setter def attr(self, v)	void setAttr(TipoR v) + getter
Propiedad calculada (R/O)		
Método que NO modifica		
Lanzar excepción de validación		

Pregunta de análisis: En C++, un const Curso no puede llamar a inscribirEstudiante() aunque sea un método perfectamente válido en tiempo de ejecución. ¿Por qué el sistema de tipos de C++ impone esta restricción, y cuál es el beneficio de diseño que esto aporta?

ACTIVIDAD 5 — Proyecto Integrador — Encapsulamiento de las 3 clases propias

Python + C++

Aplique encapsulamiento estricto a las 3 clases del Proyecto Integrador definidas en la Semana 1. Para cada clase debe entregar:

- El código Python completo con: atributos privados (__), @property para cada uno (getter y setter donde corresponda), al menos 1 propiedad calculada (sin setter), validación de invariantes con ValueError o RuntimeError descriptivo.
- El equivalente en C++17 con: atributos privados (sufijo _), getters const, setters validados que lanzan std::invalid_argument.
- El diagrama de clases UML actualizado en Draw.io mostrando: visibilidades (-, #, +), tipos de atributos, y firmas de métodos con parámetros y tipo de retorno. Exportar como PNG e insertar en el notebook.
- Commit al repositorio GitHub con el mensaje: "feat(semana2): encapsulamiento estricto e invariantes"

Criterio de diseño para esta actividad
Cada clase debe tener al menos: 3 atributos privados con invariante verificable, 1 propiedad calculada de solo lectura (derivada de otros atributos), y al menos 1 método de negocio que verifique una precondición antes de operar. Si la clase del Proyecto Integrador no cumple estos requisitos, rediseñarla ahora es mucho menos costoso que hacerlo en la Semana 9.

VI. PREGUNTAS DE REFLEXIÓN

5. Explique la diferencia entre un atributo simplemente privado (que se puede modificar sin restricción desde dentro de la clase) y un atributo con invariante (que siempre valida antes de modificarse). ¿Cuándo podría saltarse la validación justificadamente?
6. En Python, el decorador @property permite que el cliente use e.creditos = 4 (sintaxis de asignación directa) aunque internamente ejecute un setter con validación. ¿Qué ventaja de interfaz aporta esto frente al enfoque C++ de llamar setCreditos(4) explícitamente?
7. La propiedad cupos_disponibles de la Actividad 1 se calcula cada vez que se invoca. ¿Cuándo sería justificado almacenarla como atributo adicional en lugar de calcularla? ¿Qué riesgo introduce almacenarla?
8. ¿Por qué marcar los getters como const en C++ es buena práctica incluso cuando "no parece importar"? Dé un ejemplo de la Actividad 3 donde omitir const causaría un error de compilación real.

VII. RÚBRICA DE EVALUACIÓN

Criterio	Excelente (4)	Bueno (3)	Regular (2)	Insuf. (1)	Peso
@property con invariantes (Act. 1)	Todos los setters validan, propiedad calculada correcta, pruebas pasadas.	Setters validan con errores menores.	Setters sin validación.	No ejecuta.	**20%**
Composición y propiedades calculadas (Act. 2)	Docente correcto, propiedades calculadas desde estado real, pruebas pasadas.	Errores menores en propiedades calculadas.	Implementación parcial.	No ejecuta.	**20%**
C++17 const correctness (Act. 3)	Compila -Wall sin warnings, setters lanzan, getters son const.	Compila con warnings menores.	Compila sin const en getters.	No compila.	**20%**
Tabla de equivalencias (Act. 4)	Tabla completa y correcta; análisis de const profundo.	Tabla correcta; análisis superficial.	Tabla parcial.	No responde.	**15%**
Proyecto integrador encapsulado (Act. 5)	3 clases Python+C++ con invariantes, propiedad calculada, UML, commit.	3 clases Python, sin C++ o sin UML.	1-2 clases entregadas.	No entrega.	**25%**

VIII. REFERENCIAS

Meyer, B. (1997). Object-Oriented Software Construction (2nd ed.). Prentice Hall.

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Stroustrup, B. (2024). Programming: Principles and Practice Using C++ (3rd ed.). Addison-Wesley.

Python Software Foundation. (2026). Descriptor HowTo Guide. https://docs.python.org/3/howto/descriptor.html

Cppreference. (2026). const and volatile type qualifiers. https://en.cppreference.com/w/cpp/language/cv